

SafeSphere — Complete Codebase

Viva Preparation

One document covering **everything** in the `disaster-link-ai` repository: full analysis, architecture, AI/LLM/RAG internals, database, alerting, offline design, native Android app, service worker, testing, dependencies, environment variables — plus a spoken-language viva guide with 15 anticipated panel questions and a glossary. Every claim cites a real file path. Verified against source on 20 Aug 2026.

Next.js 14 App Router

Supabase + PostGIS + pgvector

LangGraph

Groq → OpenRouter → Bluesminds

XGBoost + FastAPI

Offline-first PWA

FM radio broadcast

23 languages

Capacitor + native Kotlin app

P

Team Prep Checklist

Study approach: each person should be able to explain their own section AND the 60-second pitch (Section 1) AND the full Viva Q&A (Section 18) — those two are everyone's job. Assign an owner per section, then drill the two "everyone" items together. Est. read times are rough averages.

Section	Title	Suggested owner	Est. read time
01	Project Overview (incl. 60-second pitch)	—	5 min
02	Users & Roles	—	3 min
03	Architecture & Walkthrough	—	10 min
04	Tech Stack — What & Why	—	8 min
05	Frontend Reference	—	8 min
06	Backend & API Layer	—	7 min

07	Database Design	—	7 min
08	AI / LLM / RAG System	—	12 min
09	ML Microservice	—	4 min
10	Multi-Channel Alerting	—	6 min
11	Offline-First Design	—	6 min
12	Security & Roles	—	5 min
13	Third-Party Services & Integrations	—	5 min
14	Testing & CI/CD	—	4 min
15	Native Android App	—	5 min
16	Service Worker & PWA Internals	—	4 min
17	Setup & Demo	—	6 min
18	Anticipated Viva Q&A (15 questions)	—	15 min
19	Dependency Inventory	—	4 min
20	Environment Variables	—	4 min
21	Known Gaps, Discrepancies & Assumptions	—	6 min
22	Glossary	—	4 min

110

page.tsx files

~70

API route handlers

33

Prisma models

412

component files

137+

unit tests

23

locales

29

DB migrations

61

rule-based AI fallback rules

01 Project Overview

Attribute	Value
Product name	SafeSphere
Repo	<code>disaster-link-ai</code> (package <code>disaster-response-platform</code> , <code>package.json</code>)
Track	Bharat Shakti Hackathon — "AI for Society"
Demo target	Patna, Bihar — flood-prone district; every demo flow is district-scoped
Platforms	Web PWA (Next.js) + Android (Capacitor webview + separate native Kotlin app in <code>native-android/</code>)
Backend	Supabase (PostgreSQL 15 + PostGIS + pgvector + Auth + Storage + Realtime)
LLM gateway	Groq → OpenRouter (2 keys) → Bluesminds fallback chain (<code>lib/ai/openrouter.ts</code> , <code>lib/agents/model-provider.ts</code>)
ML service	Python FastAPI + XGBoost (<code>ml_service/</code>)
Deployment	Vercel (<code>vercel.json</code>), CI on GitHub Actions (<code>.github/workflows/</code>)

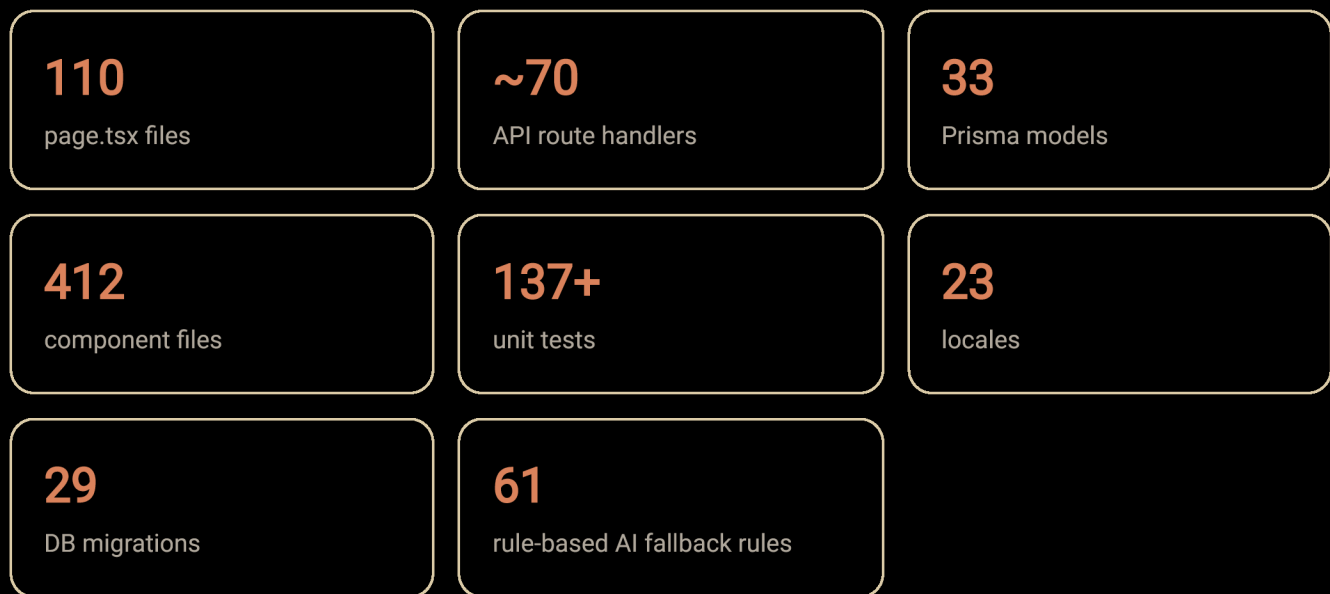
The 60-second pitch

SafeSphere is an AI-powered disaster response platform for India. It predicts flood risk with an XGBoost model, plans the response through a multi-stage pipeline, and then reaches people over *every channel that still works during a disaster* — SMS, push, WhatsApp, in-app, and FM radio for people with no smartphone or internet. Three audiences: citizens (alerts, shelters, SOS, family circle), government responders (command center, AI planner, broadcast control), and field responders (offline-first field tools). In one line: **"a Next.js web app + Supabase + Python ML service + LLM fallback chain that turns weather data into district-scoped emergency action, delivered over every channel that still works in a flood."**

C One-Page Cheat Sheet

Skimmable in under 60 seconds. Full explanations live in the sections they point to.

The 60-second pitch — SafeSphere is an AI-powered disaster response platform for India. It predicts flood risk with an XGBoost model, plans the response through a multi-stage pipeline, and then reaches people over *every channel that still works during a disaster* — SMS, push, WhatsApp, in-app, and FM radio for people with no smartphone or internet. Three audiences: citizens (alerts, shelters, SOS, family circle), government responders (command center, AI planner, broadcast control), and field responders (offline-first field tools). In one line: **"a Next.js web app + Supabase + Python ML service + LLM fallback chain that turns weather data into district-scoped emergency action, delivered over every channel that still works in a flood."**



If you remember nothing else:

- **The "AI agents" are a deterministic LangGraph assembly line** (predictor → planner → allocator → validator → communicator), not five LLMs — risk labels and plan validation are rule-based, so critical decisions like "evacuate" cannot be hallucinated (Section 8.1).
- **The LLM only powers conversational chat, behind a resilience chain:** Groq → OpenRouter (2 keys) → Bluesminds → on-device Gemma via WebLLM → 61-rule static fallback. The alerting path never touches an LLM (Sections 8.2, 8.4).
- **Offline-first because connectivity is the first casualty** — Dexie/IndexedDB + service worker + background sync let a field report succeed locally and reconcile later (Section 11).
- **District-scoped by design** — every query and every AI answer is scoped to the caller's district (`data-isolation.ts`); it is both the security model and the product model (demo = Patna).
- **What's real vs mocked (say this before asked):** the web product, schema, deterministic pipeline, FM logic, and 137+ tests are real; embeddings without a key, the ML model

artifact, Realtime transport, and most ingestion feeds are simulated or env-config only (Section 21).

02

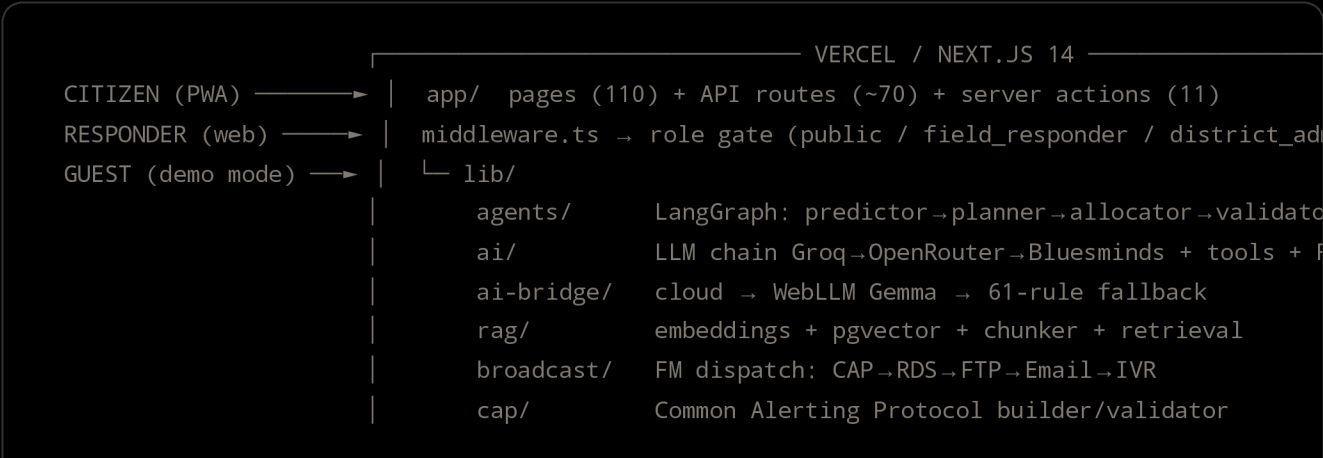
Users & Roles

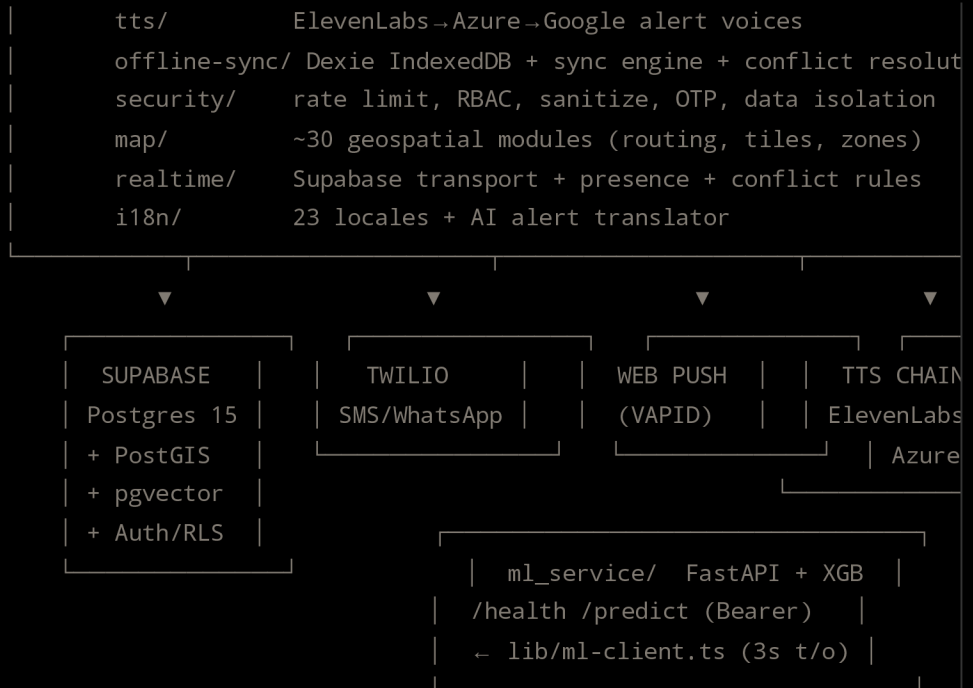
User	Role in code	What they see
Citizen	<code>public</code> (default; guest mode available)	Alerts feed, shelters on a MapLibre map, SOS, family circle, evacuation routes, 23-language support – <code>((public)/</code> and <code>((app)/</code> route groups
Government responder	<code>district_admin</code> / <code>super_admin</code>	Command center, AI planner chat, agent orchestration, resource inventory, FM broadcast queue + approvals, TTS voice preview – <code>((admin)/</code> , <code>((gov)/</code>
Field responder	<code>field_responder</code>	Lite/offline kits, reports, check-ins – <code>((field)/</code>
Platform admin	<code>super_admin</code>	User management, audit logs, system health, bulk operations, district config
Guest	<code>guest_mode</code> cookie	Full demo experience, zero signup – hackathon demos (<code>middleware.ts</code> , <code>app/actions/auth.ts</code>)

The role enum lives in `prisma/schema.prisma` (`UserRole`: `public | field_responder | district_admin | super_admin`). `middleware.ts` also tolerates a legacy `viewer` role.

03

Architecture & Walkthrough





One scenario traced end-to-end: "a flood alert reaches a citizen's phone"

- Weather data arrives.** Vercel cron (`vercel.json`: `0 0 * * *`) hits `app/api/cron/ingest/route.ts` (bearer-locked with `CRON_SECRET`) → `lib/data-ingestion/fetcher.ts` pulls rainfall from OpenWeatherMap (the only fully wired external feed) [△ See Known Gaps \(#10\)](#) → `WeatherData` row.
- Risk is predicted.** `app/api/predict/route.ts` → `lib/ml-client.ts` POSTs 4 features (`cumulative_rainfall_72h`, `river_level_trend`, `soil_saturation_index`, `elevation_m`) to `ml_service/api.py`. XGBoost returns `Low/Medium/High/Critical`, mapped to `Safe/Watch/Warning/Evacuate`; a `FloodPrediction` row is stored. [△ See Known Gaps \(#6\)](#)
- An alert is created.** `server/services/alert-engine.ts` (6-hour dedup) or an admin via `app/actions/alerts.ts` creates an `AlertLog` row; channels from `DEFAULT_CHANNELS = sms | in_app | push`; text templated by `lib/alerts/template-parser.ts`.
- Channels fire.**
 - SMS** — `lib/alerts/twilio-client.ts`
 - Push** — `lib/alerts/web-push.ts` + `server/services/push-notifier.ts`; service worker `worker/index.js` receives it with the tab closed
 - In-app** — `app/api/alerts/feed`; cached in Dexie `alerts` table (`lib/offline-sync/db.ts`)

- FM radio (escalation) — if an `AlertRuleFm` matches at severity

`warning`/`critical`: `lib/broadcast/auto-trigger.ts` → `fm-dispatcher.ts`
(Section 10)

5. Citizen sees it. Push banner; dashboard alert (`((app)/dashboard)`); flood zones light up on the map (`lib/map/flood-geojson.ts`, `hazard-geojson.ts`). Offline? Already in IndexedDB.

How to explain this in a viva: "The whole system is one Next.js app talking to Supabase, with three satellite pieces: a Python ML service that predicts flood risk from 4 hydrology features, a chain of LLM providers for the AI planner chat, and a set of delivery channels that fire in parallel so an alert survives even if one channel is down. The critical insight: the *alerting path* is deterministic and rule-based, while the *conversational AI* is where the LLM lives — we never let an LLM decide whether to evacuate a district."

04 Tech Stack — What & Why

Technology	What it is	Used here for	Why over alternatives
Next.js 14 + React 18	React framework with file-based routing & SSR	All pages, API routes (<code>((app/api/**))</code>), server actions (<code>((app/actions/))</code>)	One codebase for server + client; secrets stay server-side; <code>next dev --turbo</code> for speed
TypeScript 5	Typed JS	Everything	Catches contract errors at build time; Prisma types flow into components
Tailwind CSS 3.4	Utility-first CSS	All styling (<code>((globals.css))</code> + CSS vars)	Fast to build; <code>daisyui</code> is in <code>package.json</code> but unused <code>unused</code>
MapLibre GL 4.7 + react-map-gl 7.1	Open-source map renderer	All maps (flood zones, shelters, routes, fleet)	No Mapbox token; offline tile caching (<code>((lib/map/offline-map-manager.ts))</code>)

@turf/turf 7.4	Geospatial math library	Distances, buffers, nearest-shelter, coverage	Small pure-JS geometry complementing PostGIS
Supabase (PostgreSQL 15)	Hosted Postgres + Auth + Storage + Realtime	The whole backend	PostGIS and pgvector in one DB; free tier; Realtime
Prisma 6.19	Type-safe ORM	All DB access (<code>server/prisma.ts</code>)	Type safety across 33 models; (<code>postinstall: prisma generate</code>); (<code>DIRECT_URL</code>) for migrations
PostGIS	Postgres geospatial extension	FM (<code>coverage_area</code>) (<code>lib/fm/find-stations.ts</code>), nearest shelter, hazard polygons	SQL-level spatial indexing – one indexed query, not a JS loop
pgvector	Postgres vector extension	Semantic SOP search (RAG, Section 8)	No separate vector DB; HNSW index beside the data
LangChain + LangGraph	LLM orchestration + graph framework	Response pipeline (<code>lib/agents/graph.ts</code>) + RAG chunking	Fixed, inspectable graph – auditable per-node state
LLM fallback chain	Groq → OpenRouter → Bluesminds	(<code>lib/ai/openrouter.ts</code>), (<code>lib/agents/model-provider.ts</code>)	No single point of failure for the "AI advisor"
XGBoost + FastAPI	Gradient-boosted trees + Python API framework	(<code>ml_service/</code>) – flood risk (4 features → 4 classes)	Trees beat NNs on small tables; CPU training; interpretable importance
Twilio 6	SMS/WhatsApp API	(<code>lib/alerts/twilio-client.ts</code>), signature-verified webhooks	Standard gov alerting channel; trial credits for demos
Web Push (VAPID)	Browser push protocol	(<code>lib/alerts/web-push.ts</code>) + (<code>worker/index.js</code>)	Free; works with the tab closed
Dexie 4 + next-pwa/Workbox	IndexedDB wrapper + SW tooling	Offline-first storage (<code>lib/offline-sync/</code>) +	Typed IndexedDB; per-route caching strategies


```
shell (bharat-shakti-shell-  
v1)
```

Capacitor 8.5	Web → native wrapper	(capacitor.config.ts) → Android (com.safesphere.app)	Instant APK from the same web code
@mlc-ai/web-llm 0.2	In-browser LLM runtime	On-device Gemma-2B fallback (lib/ai-bridge/webllm-provider.ts)	AI keeps working with zero connectivity

The 4 most important choices, in prose

1 · Next.js 14 with API routes + server actions. Everything — pages, REST endpoints, and form mutations — lives in one app (`app/`). ~70 API routes keep secrets server-side and one auth middleware (`middleware.ts`) gates everything. Server actions (11 files in `app/actions/`) handle form-style mutations without a separate API layer.

2 · Supabase as Postgres + PostGIS + pgvector in one place. Deliberately no separate vector or spatial database. PostGIS gives FM-station coverage checks (`lib/fm/find-stations.ts`) and flood-zone polygons; pgvector gives semantic search over SOP documents (`lib/rag/vector-search.ts`, cosine `<=>`). One connection string, one backup, one migration pipeline — Prisma sees it all as ordinary models.

3 · The LLM fallback chain. `lib/ai/openrouter.ts` builds Groq (free) → OpenRouter (primary) → OpenRouter (backup) → Bluesminds, probing each with a 1-token request and caching health with a TTL. `lib/agents/model-provider.ts` wraps the same idea with `withFallbacks`. During development we actually hit a 404'd model and a zero-credit key — the incidents are documented in the file header. A disaster tool whose AI dies when one API key runs out is useless.

4 · XGBoost + FastAPI microservice. Flood risk is a small tabular problem: 4 engineered features (`ml_service/api.py`), 4 risk classes, `MIN_ROWS = 20` training guard (`train_model.py`). Trees train in seconds on a laptop, need no GPU, and expose feature importances — defensible: an officer can see *why* a district moved to "High". Bearer-key protected (`ML_API_KEY`); the web app calls it via `lib/ml-client.ts` with a 3-second timeout.

How to explain this in a viva: "We picked the stack for three reasons: one language across the whole product, one database that does relational + spatial + vector work, and zero single points of failure in the AI path. The ML service is deliberately separate from the web app because it needs its own model lifecycle – retraining and versioning – and the LLM chain exists because a disaster response tool that dies when one API key runs out is useless."

05 Frontend Reference

Route map (110 `page.tsx`, 12 `layout.tsx`)

Route group	Purpose	Representative pages
<code>/</code> root	Landing, portal	<code>(public)/landing</code>
<code>(public)</code>	Citizen-facing content	landing, alerts, shelters, SOS, family, gov info
<code>(auth)</code>	Authentication	login, register, otp, role picker
<code>(dashboard)</code>	Logged-in shell	settings (ai, notifications, map, privacy, org, integrations, shift-schedule), directory, family, SOS, shelters
<code>(field)</code>	Field-responder tools	offline kits, lite mode, reports, check-ins
<code>(admin)</code>	District/super admin	command center, resources, alerts, evacuations, FM broadcast queue, TTS voice preview, users, audit, bulk ops, system health, analytics
<code>(gov)</code>	Government dashboard	AI planner, agent orchestration, triage, inventory, knowledge base
<code>(app)</code>	Citizen app (mobile-first)	dashboard, emergency, lite, profile-setup
Top-level	Utility	<code>~offline</code> (SW fallback), <code>403</code> , <code>debug</code> , <code>demo</code> , <code>directory</code> , <code>download</code> , <code>emergency</code> , <code>family</code> , <code>gov</code> , <code>lite</code> , <code>profile-setup</code> , <code>public</code>

UI stack

Concern	Technology	Evidence
Styling	Tailwind CSS 3.4 + CSS variables (DaisyUI unused)	<code>globals.css</code> , <code>package.json</code>
Components	shadcn/ui convention — 4 primitives vendored: <code>button</code> , <code>card</code> , <code>badge</code> , <code>avatar</code> (Radix <code>avatar</code> + <code>slot</code>)	<code>components/ui/</code>
Icons / Animation	<code>lucide-react</code> / <code>framer-motion</code>	<code>package.json</code>
Tables / Charts	<code>@tanstack/react-table</code> / <code>recharts</code>	<code>package.json</code>
Forms / Validation	<code>react-hook-form</code> + <code>zod</code> + <code>@hookform/resolvers</code>	<code>package.json</code>
Toasts / Markdown	<code>react-hot-toast</code> / <code>react-markdown</code> + <code>remark-gfm</code>	<code>package.json</code>
QR / Theme	<code>qrcode.react</code> + <code>html5-qrcode</code> / <code>next-themes</code>	<code>package.json</code>

Component inventory (28 subdirectories, ~412 files under `components/`)

Organized by domain: `admin/` (broadcast queue/monitor/history, FM station manager, TTS voice preview, users, audit, analytics, system health), `agents/` (agent configuration panel, orchestration UI), `citizen/`, `field/`, `gov/` (planner, triage, inventory, knowledge base), `map/` (map shells, layer toggles, offline tiles), `ui/` (the 4 shadcn primitives), plus alerts, shelters, resources, evacuations, sos, family, chat, settings and shared layout widgets. Map-heavy screens load via `next/dynamic({ ssr: false })`.

Hooks (25 hooks + 6 hook test files)

Notable: `useAI` / `useDisasterChat` (AI planner), `useTextToSpeech` (alert voice), `useWakeLock` (keep screen on during ops), `useBatterySaver` (battery-aware behavior), `useShakeToSOS` (mobile SOS trigger), plus offline/sync, notification settings, language, theme and map-settings hooks. State is React context + hooks — deliberately no external store.

Mapping & GIS

Concern	Technology	Evidence
Renderer	MapLibre GL <code>^4.7.1</code> + <code>react-map-gl</code> <code>^7.1.9</code>	<code>package.json</code>
Draw / Geometry	<code>@mapbox/mapbox-gl-draw</code> / <code>@turf/turf</code>	<code>package.json</code>
Layers	flood zones, hazard zones, shelters, road closures, evacuation routes, fleet	<code>lib/map/*</code> (~30 modules, e.g. <code>flood-geojson.ts</code> , <code>hazard-geojson.ts</code> , <code>gov-map-layers.ts</code>)
Offline tiles	Dexie tile store + <code>offline://</code> protocol	<code>lib/map/offline-map-manager.ts</code> , <code>offline-protocol.ts</code> , <code>offline-clustering.ts</code>
Routing	OSRM public + optional GraphHopper / openrouteservice	<code>lib/map/routing.ts</code> , <code>smart-routing.ts</code> , <code>route-safety.ts</code> , <code>citizen-evacuation-route.ts</code>

i18n – 23 languages

Custom lightweight i18n (no `next-intl`): 23 locales – English + 22 Eighth-Schedule Indian languages – lazy-loaded via `lib/i18n/LanguageContext.tsx` / `locales.ts`. Alerts get translated for SMS by `lib/i18n/ai-translator.ts` (`translateAlertForSMS`).

06

Backend & API Layer

API routes by group (~70 handlers under `app/api/**/route.ts`)

Group	Routes	Purpose
AI & agents	<code>agents/orchestrate</code> , <code>chat</code> , <code>chat/sessions</code> , <code>predict</code> , <code>rag/search</code>	Multi-agent pipeline, streaming chat (AI SDK v7 <code>streamText</code>), ML prediction, RAG

Alerts	alerts/create , alerts/feed , alerts/preferences , alerts/audit , alerts/audio/*	Alert lifecycle, preference routing, TTS audio
Broadcast/FM	broadcast/fm/dispatch , auto-trigger , queue , approvals , history , stations , compliance	FM dispatch, rule auto-trigger, approval workflow, station CRUD
CAP	cap/generate , cap/validate , cap/serve	Common Alerting Protocol documents
Citizen	sos , family , shelters/nearest , shelters/routes , evacuations/plans , missing-persons , crowdsourced/report	SOS events, family circle, nearest shelter, evacuation
Data ingestion	cron/ingest (Vercel cron, CRON_SECRET), cron/audio-retention	Weather/hydrology ingestion, audio cleanup
Maps	map/export , map/flood-zones , map/hazard-zones , map/road-closures , map/measurements	GeoJSON export, layer data
Resources	resources/inventory , resources/requests , resources/allocate , allocations/optimize	Inventory + allocation optimizer (lib/allocation/optimizer.ts , fleet-optimizer.ts)
Push/SMS/WhatsApp	push/subscribe , push/send , webhooks/sms , webhooks/whatsapp , whatsapp/inbound , sms/commands	VAPID push; Twilio inbound with signature verification
TTS	tts/generate	Alert voice generation

System

`settings/*`, `demo/reset`,
`debug`, `download/apk`,
`health`

Settings, demo reset, APK download

Server actions (`app/actions/`, 11 files)

`auth.ts` · `shelters.ts` · `resources.ts` · `reports.ts` · `alerts.ts` · `admin.ts` ·
`documents.ts` · `assignments.ts` · `preferences.ts` · `simulation.ts` · `whatsapp.ts`

Middleware (`middleware.ts`)

- Cookie-based session/role gate: `role`, `sandbox`, `guest_mode`, `district` cookies.
- `PUBLIC_ROLE = "public"`, `GOV_ROLES`, `PROTECTED_PATHS`, `DASHBOARD_PATHS` lists; strict admin-route enforcement; 403 page; district crossover guard.

Auth model (`app/actions/auth.ts`, `lib/security/otp.ts`)

- **OTP-first, passwordless:** `sendOTP` / `verifyOTP` via the GetOTP provider.
- Demo bypass: seeded demo OTP + guest mode (`GUEST_COOKIE`, `guest_mode`).
- Cookies: `httpOnly`, `sameSite: "lax"`, `secure`, `path: "/"`. Supabase Auth optional — fully usable in demo mode.

How to explain this in a viva: "One app, one middleware, one auth model. API routes are the integration layer — they keep secrets on the server and apply the same role checks as the pages — while server actions handle the form-level mutations. Everything is cookie-session based with an OTP path, and a guest-mode cookie lets panels click through the entire product without signup."

07 Database Design

Platform

- **Supabase PostgreSQL 15** with extensions `postgis`, `vector`, `pgcrypto` (`supabase/migrations/0002_postgis`, `0003_pgvector`).

- Prisma 6.19.3; connection pair `DATABASE_URL` (pooler) + `DIRECT_URL` (migrate/studio) per `.env.example`.

All 33 models, by logical cluster

Cluster	Models	What it's for
Users & identity	<code>User</code> (enum <code>UserRole</code>), <code>FamilyContact</code> , <code>PushSubscription</code>	Who's using the app; family circle for SOS; VAPID subscriptions pruned by <code>server/services/push-notifier.ts</code>
Disaster intelligence	<code>DisasterEvent</code> , <code>FloodPrediction</code> , <code>WeatherData</code> , <code>DataSource</code> , <code>EmergencyDocument</code> , <code>RoadClosure</code> , <code>CrowdsourcedReport</code>	Everything the system <i>knows</i> : weather in, predictions, SOP docs for RAG, closures, citizen reports
Response planning	<code>EmergencyPlan</code> , <code>EvacuationPlan</code> , <code>PlanFeedback</code> , <code>Resource</code> , <code>ResourceRequest</code> , <code>ResourceMovement</code> , <code>ResourceAllocation</code> , <code>MissingPerson</code> , <code>CasualtyRecord</code>	What we'll do, with what, who's missing — feeds the agent pipeline
Alerts & broadcast	<code>AlertLog</code> , <code>AlertRule</code> , <code>AlertTemplate</code> , <code>AlertAudio</code> , <code>AlertRuleFm</code> , <code>FmApprovalRequest</code> , <code>FmStation</code> , <code>FmBroadcastLog</code> , <code>CapAlert</code>	Delivery layer; FM rules + approvals (<code>lib/broadcast/auto-trigger.ts</code>); CAP docs; TTS audio in <code>alert-audio</code> bucket
AI & chat	<code>ChatSession</code> , <code>ChatMessage</code> , <code>ToolCallLog</code>	Conversation persistence + audit trail of every AI tool call
Operations	<code>AuditLog</code>	Immutable admin-action trail (<code>lib/admin/audit-logger.ts</code>)

README says "28 models" — the actual schema has 33. Say 33 if asked.

Migrations (`supabase/migrations/0001-0029`)

- 0002 PostGIS, 0003 pgvector, RLS policies in 0017/0020/0022, role CHECK constraint in 0023.

- **missing** Migration **0018** is absent (0017 → 0019) — verify continuity before production.

Why PostGIS and pgvector were necessary, not optional

- **PostGIS** answers genuinely spatial questions at SQL speed: "which FM stations' `coverage_area` polygons overlap this district?" (`ST_Intersects`, `lib/fm/find-stations.ts`), "nearest shelter within 5 km" (`lib/map/nearby-shelters.ts`), flood polygons (`lib/map/flood-geojson.ts`). In JS this would mean loading every geometry into memory.
- **pgvector** keeps embeddings *next to* the source documents instead of a second vector DB — retrieval and source-of-truth are the same row.

How to explain this in a viva: "We chose one database that does three jobs — relational, spatial, and vector — because disaster response is a coordination problem and forcing joins across three databases during a flood would be a nightmare. PostGIS answers 'who covers what area' at SQL speed and pgvector lets us do document search without a second vector store. Both decisions were about operational simplicity under stress."

08 AI / LLM / RAG System

8.1 What "multi-agent" means here — no jargon

Imagine an assembly line, not a team of genius robots. A box comes in labeled "district = Patna, disaster = flood, rainfall = 210 mm". Five workers stand in a line, each doing one simple job:

1. **Worker 1 (predictor)** stamps a risk label: SAFE / WATCH / HIGH / CRITICAL.
2. **Worker 2 (planner)** writes the plan: evacuate these blocks, open these shelters.
3. **Worker 3 (allocator)** decides which boats, buses, and kits go where.
4. **Worker 4 (validator)** checks the plan isn't nonsense.
5. **Worker 5 (communicator)** runs only *after a human approves* — it fires the alerts.

That assembly line is a **LangGraph** — a graph where each node is one worker and you can inspect the state at every step. In code: `lib/agents/graph.ts` defines `START →`

`predictorNode → plannerNode → allocatorNode → validatorNode → END`, with `communicatorNode` invoked externally once approval happens (`app/api/agents/orchestrate/route.ts`). Statuses: `predicting → planning → allocating → pending_approval → communicating → resolved`.

Honesty note The README describes named LLM agents – Flood Analyst, Shelter Manager, Resource Coordinator, Evacuation Planner, Communications Officer. The code implements the same *concepts* as **deterministic rule-based nodes** (`lib/agents/nodes/intelligence-nodes.ts`): risk is computed by keyword/feature matching, not by an LLM. The LLM's job is the conversational planner chat (`app/api/chat/route.ts`) and its tools. Own this decision in a viva: deterministic logic can't hallucinate a risk label; the LLM is layered where open-ended reasoning is actually needed.

Worker (code name)	Receives	Decides	Hands off
<code>predictorNode</code> (<code>lib/agents/nodes/intelligence-nodes.ts</code>)	Event: district, disaster type, rainfall, river level	Risk label <code>SAFE/WATCH/HIGH/CRITICAL</code>	Stamped state → planner
<code>plannerNode</code> (same file)	Risk label + district context	Emergency plan: actions, shelters, evacuation steps	Plan → allocator
<code>allocatorNode</code> (<code>lib/agents/nodes/action-nodes.ts</code>)	Plan	Resources/fleet per task	Allocation → validator
<code>validatorNode</code> (same file)	Allocation	Plan feasible? (→ <code>pending_approval</code>)	Approved plan → communicator
<code>communicatorNode</code> (external, API-triggered)	Approved plan	Fire SMS / push / in-app alerts	Channels (Section 10)

8.2 Provider chain (resilience core)

Layer	File	Details
-------	------	---------

AI SDK v7 + LangChain	<code>package.json</code>	<code>ai ^7.0.55</code> , <code>@ai-sdk/openai</code> , <code>@langchain/openai</code> , <code>@langchain/langgraph</code>
Emergency-planner chain	<code>lib/ai/openrouter.ts</code>	Probe-based (1-token) health check with TTL cache; Groq → OpenRouter primary → OpenRouter backup → Bluesminds ; AI SDK v7 <code>stopWhen</code> fix; leaked-key regression tests
Agent graph models	<code>lib/agents/model-provider.ts</code>	<code>withFallbacks([...]); model</code> <code>~anthropic/claude-sonnet-latest</code>
Streaming chat	<code>app/api/chat/route.ts</code>	<code>streamText</code> + <code>toUIMessageStreamResponse</code> , <code>stopWhen(isStepCount(6))</code> , <code>maxOutputTokens:</code> <code>2048</code> , rate limit 5/min/IP (<code>lib/security/ai-rate-</code> <code>limit.ts</code>), tools: flood, shelters, resources, evacuation, emergency plans (<code>lib/ai/tools/</code>)

8.3 RAG — plain version, then the actual pipeline

Plain version: RAG means "before the AI answers, go fetch the actual documents that contain the answer and paste the relevant bits into the prompt." It's quoting from the handbook in front of it instead of reciting from memory.

1. **Ingest PDFs** — SOPs, NDMA guidelines, district DMPs → `lib/rag/chunker.ts` splits with `RecursiveCharacterTextSplitter` (1000-char chunks, 200-char overlap); PDFs via `pdf-parse`.
2. **Embed** — `lib/rag/embeddings.ts` → OpenAI `text-embedding-3-small`, 1536-dim, LRU-cached. `Mock fallback` without a key — demo still works, semantic search degrades to keyword. [△ See Known Gaps \(#5\)](#)
3. **Store** — vectors + text in a `vector` column; `lib/rag/vector-search.ts` queries cosine `<=>` over HNSW, optional `districtFilter`.
4. **Retrieve** — `lib/retrieval/retrieve.ts`: two-tier search (vector first, keyword fallback) so retrieval never returns empty.
5. **Inject** — retrieved SOP text dropped into prompts as `{SOP_CONTEXT}` (`lib/agents/nodes/intelligence-nodes.ts`) and via `buildKnowledgeContext` (`app/api/chat/route.ts`).

8.4 Offline / local AI (AIBridge, Phase 9)

- `lib/ai-bridge/`: routes cloud → on-device WebLLM Gemma-2B (`gemma-2b-it-q4f32_1-MLC`) → 61-rule deterministic fallback (`rule-based-fallback.ts`).
- Connectivity monitor (`connectivity.ts`): `navigator.onLine` + Supabase heartbeat; confidence scoring (`confidence.ts`), token estimation (`estimate-tokens.ts`), worker provider (`worker-provider.ts`).
- Legacy path: `lib/ai/AIBridge.ts`, `LocalLLMProvider.ts`, `FallbackEngine.ts`; model download + integrity into Dexie `gemmaModel` table (`lib/offline-sync/model-store.ts`, `model-integrity.ts`).

8.5 Five likely viva questions about the AI layer

Q. Why not just call one LLM directly instead of LangGraph?

The response-planning flow must be *auditable and deterministic in its critical decisions*. LangGraph gives a fixed pipeline where every node is a single job, state is inspectable at each step, and — crucially — risk labels and plan validation are rule-based, so no LLM can hallucinate "evacuate" or "all clear". The LLM is reserved for open-ended chat where tools (`lib/ai/tools/`) keep it grounded in DB facts.

Q. How do you prevent the AI from hallucinating wrong emergency advice?

Four layers: (1) the chat model must call tools to see real data; (2) RAG context is injected via `{SOP_CONTEXT}` so answers quote documents; (3) every response is district-scoped by `assertDistrictAccess/enforceDistrictScope` (`app/api/chat/route.ts`); (4) every tool call is logged to `ToolCallLog` (`lib/security/tool-logger.ts`).

Q. What happens if all your LLM providers are down at once?

The chain steps down Groq → OpenRouter → Bluesminds, then AIBridge hands off to on-device Gemma via WebLLM, then to the 61-rule fallback. The *alerting path never depends on an LLM at all* — it's deterministic (`lib/broadcast/auto-trigger.ts`). Only conversational chat degrades.

Q. Why XGBoost for flood prediction and not a neural network?

Flood risk from hydrology is a small tabular problem — 4 engineered features, thousands of rows at most. Boosted trees train in seconds on CPU, need no GPU or big

dataset, and give interpretable feature importances — "why did the district go to High?" is answerable with tree models, not with a neural net.

Q. How does RAG know which documents are relevant to a query?

Every chunk is embedded to 1536 dimensions; the query is embedded the same way; pgvector returns nearest chunks by cosine similarity (`<=>`, HNSW). Top-k chunks feed the prompt; without keys, keyword search takes over (`lib/retrieval/retrieve.ts`) so retrieval never fails silently.

How to explain this in a viva: "We built the AI as a layered system. The response pipeline is a deterministic LangGraph — think assembly line, not autonomous agents — because safety-critical decisions like 'evacuate' must be explainable and can never be hallucinated. The LLM is used where open-ended reasoning is needed, grounded by RAG over SOP documents and forced through tools that can only see real database data. Every layer has a fallback, down to a rule-based engine, so the system degrades gracefully instead of dying."

09 ML Microservice (`ml_service/`)

[△ See Known Gaps \(#6\)](#) — the trained model artifact is not committed (`ml_models/` is `.gitkeep` only), and without the service running, `/api/predict` falls back to mock labels. Details in Section 21.

File	Role
<code>api.py</code>	FastAPI: <code>/health</code> + <code>/predict</code> ; Bearer-key protected (<code>ML_API_KEY</code>); 4 features: <code>cumulative_rainfall_72h</code> , <code>river_level_trend</code> , <code>soil_saturation_index</code> , <code>elevation_m</code> ; returns class + probabilities; <code>RISK_LABELS = Low/Medium/High/Critical</code>
<code>train_model.py</code>	XGBoost 4-class classifier; <code>MIN_ROWS = 20</code> early-abort; saves <code>flood_xgboost_model.pkl</code>
<code>requirements.txt</code>	fastapi, uvicorn, pandas, numpy, scikit-learn, xgboost, pydantic, joblib, pypdf, openai, httpx

`ML_DATABASE/`

Training corpus: ~50 PDFs (CWC, IMD, NDMA, NDRF, flood reports),

`Flood_Prediction.ipynb`, decks

`ml_models/``.gitkeep` only — model artifact expected at runtime`not committed`

Client bridge: `lib/ml-client.ts` POSTs features to `ML_SERVICE_URL`
(`http://127.0.0.1:8000`), 3-second timeout; maps `Low/Medium/High/Critical` →
`Safe/Watch/Warning/Evacuate`. Wrapped for the UI by `app/api/predict/route.ts`.

10 Multi-Channel Alerting

Why five channels instead of one

The audience during a Bihar flood: someone with a smartphone and data (push + app), someone with only a basic phone (SMS), someone using WhatsApp as their only internet app (WhatsApp), and a whole village with *no phone at all* (FM radio). One channel = guaranteed partial reach. The alert engine (`server/services/alert-engine.ts`) fans out rather than picks one: `DEFAULT_CHANNELS = sms | in_app | push`, 6-hour dedup, templated copy (`lib/alerts/template-parser.ts`), and regional-language SMS translation (`lib/i18n/ai-translator.ts`).

Channels

Channel	Implementation	Notes
SMS	<code>lib/alerts/twilio-client.ts</code> , <code>lib/sms/twilio-webhook.ts</code>	Signature-verified inbound; SMS commands (<code>lib/sms/sms-commands.ts</code>)
WhatsApp	<code>lib/sms/whatsapp-commands.ts</code> , <code>app/api/whatsapp/inbound</code>	Command-driven inbound
Web Push	<code>lib/alerts/web-push.ts</code> , <code>server/services/push-notifier.ts</code> , <code>worker/index.js</code>	VAPID; subscription pruning; periodic background sync
In-app	<code>app/api/alerts/*</code> + Dexie cache	Offline-capable

The FM radio path (the unusual one)

1. **Trigger** — lib/broadcast/auto-trigger.ts normalizes risk labels (Safe/Watch/Warning/Evacuate or low/mod/high/critical) into a severity tier (warning|critical), matches the best AlertRuleFm (exact district first, then catch-all 'all'), rate-limits to MAX_BROADCASTS_PER_MINUTE = 5.
2. **Human gate** — if rule.auto_broadcast is off, an FmApprovalRequest lands in the admin queue; DEFAULT_AUTO_APPROVE_SECONDS = 180 — unactioned requests auto-approve after 3 minutes (lazily via autoApproveExpired()); in a real flood you'd rather broadcast than wait.
3. **CAP document** — lib/cap/cap-builder.ts + cap-service.ts generate Common Alerting Protocol XML; cap-hash.ts makes it idempotent; cap-validator.ts validates; cap-templates.ts + districtHeadline / resolveCapPreset provide copy.
4. **Dispatch** — lib/broadcast/fm-dispatcher.ts finds covering stations via PostGIS (lib/fm/find-stations.ts), type priority (air > private > community), then runs the strategy chain (cap_api → rds → ftp → email) (strategy-selector.ts), escalating to IVR phone calls (fm-ivr-fallback.ts) when no digital path succeeds. Max 3 attempts, 2-minute backoff; testMode = dry-run report.
5. **Station side** — CAP API call, RDS text (rds-encoder.ts, rds-text.ts), FTP drop (basic-ftp), or email to the studio inbox (email-studio.ts); every attempt recorded in FmBroadcastLog; lib/broadcast/compliance.ts keeps the audit trail; human-readable dispatch reports via report.ts / report-html.ts.

Alert voice (TTS, Phase 2)

- Chain: ElevenLabs → Azure Neural → Google Cloud (lib/tts/providers.ts); mono MP3 128 kbps, 20 s timeout, estimateDurationSec.
- Templates + languages: lib/tts/templates.ts, languages.ts; audio in Supabase alert-audio bucket (lib/tts/audio-store.ts); retention cron app/api/cron/audio-retention.
- Supporting: lib/tts/alert-voice-generator.ts, beep.ts (alert tones), lib/audio/alert-announcer.ts.

How to explain this in a viva: "One channel is a decision to fail. We fan alerts out to SMS, push, WhatsApp, in-app, and FM radio — and the FM path is the only one that reaches people with no phone at all. It's a real pipeline: rule-based trigger, CAP alert generation, a strategy chain from a station's CAP API down to FTP and email, then IVR escalation if all digital paths fail. There's a human approval step that auto-approves after three minutes — in a flood, 'broadcast now, ask forgiveness later' is the correct bias."

11 Offline-First Design

Why offline, in plain terms

A flood doesn't knock out power and towers *later* — it knocks them out *first*. The people who most need the system have the worst connectivity. So the app is **offline-first by design**: a PWA (`next-pwa`), custom worker (`worker/index.js`), offline shell (`bharat-shakti-shell-v1`) with all operational data mirrored into Dexie/IndexedDB.

Dexie schema — database `disasterlink-offline` v2

Table	Contents
<code>predictions</code> , <code>alerts</code> , <code>routes</code> , <code>resources</code> , <code>weather</code> , <code>profiles</code> , <code>maps</code> , <code>knowledge</code> , <code>shelters</code> (<code>"id, district"</code>)	Operational data mirrored for offline use (<code>lib/offline-sync/db.ts</code>)
<code>metadata</code> , <code>chatHistory</code>	Sync bookkeeping + AI chat history
<code>mapTiles</code>	Cached map tiles for <code>offline://</code> rendering
<code>gemmaModel</code>	Downloaded WebLLM model binary + integrity info

Trace: a citizen report submitted with no signal

1. The field responder opens the app in `((field)/)` lite mode. With `navigator.onLine === false` (or the AIBridge connectivity monitor detecting a dead Supabase heartbeat — `lib/ai-bridge/connectivity.ts`), the app writes the report into Dexie immediately. The UI never waits on the network.
2. Background sync (registered in `worker/index.js`) notices connectivity is back.
3. `lib/offline-sync/sync-engine.ts` drains the local queue to the server; `lib/realtime/conflict.ts` resolves conflicts (timestamp-based last-write-wins) so offline and online edits don't silently lose data.
4. The server records the `CrowdsourcedReport`; the device gets the acknowledgment on the next sync pass.

Supporting pieces: `storage-pressure.ts` (evicts before browser quota bites), `quota.ts`/`eviction.ts` (budgets), `model-integrity.ts` (Gemma model verification), `sw-sync.ts` (service-worker sync bridge), `lib/offline/cache.ts`, `step-directions.ts`, `lib/field-offline.ts`, `offline-context/` (context builder for offline AI prompting).

How to explain this in a viva: "We built the UI to never depend on the network — writes go to IndexedDB first and sync in the background, and the service worker caches the app shell and API responses network-first. The reason is simple: we simulated the failure we're trying to fix. If the app needs internet to report a flood, it's useless exactly when it's needed most. Offline-first is the product's whole reason to exist, not a bolt-on."

12 Security & Roles

The story: citizen vs. government admin vs. guest

A citizen (`public`) can read alerts, shelters, SOS, family circle — but cannot reach admin pages. `middleware.ts` holds `PROTECTED_PATHS`/`DASHBOARD_PATHS`; `lib/security/require-role.ts` re-checks the role server-side on every protected request (the cookie is a hint, the Supabase profile is the truth). All data is district-scoped via `lib/security/data-isolation.ts`, and the AI chat applies `assertDistrictAccess` + `enforceDistrictScope` (`app/api/chat/route.ts`).

A government admin (`district_admin`/`super_admin`) gets the command center, FM controls, resources. The broadcast route is the strictest:

`app/api/broadcast/fm/dispatch/route.ts` requires `BROADCAST_ROLES`, rate-limits to 5 broadcasts/minute/IP, and supports `testMode` dry-runs. `super_admin` additionally gets user management and audit logs.

A guest gets the `guest_mode` cookie (`app/actions/auth.ts`) – full access to demo-scoped data with zero signup. Deliberate for the hackathon: panels click through the whole product; `lib/demo/` (`demo-mode.ts`, `seeder.ts`, `reset-scenario.ts`) keeps the demo self-contained.

Security controls, and why each matters for a disaster app specifically

Control	Where	Why it matters here
Rate limiting (in-memory sliding window, <code>MAX_TRACKED_IDENTIFIERS = 10_000</code> ; AI chat 5/min/IP; broadcast 5/min)	<code>lib/security/rate-limit.ts</code> , <code>rate-limiter.ts</code> , <code>ai-rate-limit.ts</code> , <code>app/api/broadcast/fm/dispatch/route.ts</code>	One misbehaving client must not take down the alert API – rate limits protect the <i>channel</i>
XSS sanitization + PII redaction (<code>[REDACTED]</code>)	<code>lib/security/sanitize.ts</code>	Crowdsourced reports must never inject scripts into an admin's command center; victims' phone numbers never reach logs
Twilio webhook signature verification	<code>lib/sms/twilio-webhook.ts</code>	Only genuine Twilio calls may create alert actions
District data isolation	<code>lib/security/data-isolation.ts</code> , <code>middleware.ts</code>	Patna's data invisible to a Gaya admin – leaks during a disaster are operational failures

Server-only secrets	env vars + <code>lib/supabase/server.ts</code>	The service-role key must never reach the client
Cookie hardening	<code>app/actions/auth.ts</code> (<code>httpOnly</code> , <code>sameSite lax</code> , <code>secure</code>)	Session tokens not readable by scripts
AI tool-call audit log	<code>lib/security/tool-logger.ts</code> → <code>ToolCallLog</code>	If the AI recommends something, there's a record of what data it used
Cron bearer token (<code>CRON_SECRET</code>)	<code>app/api/cron/ingest/route.ts</code>	Nobody can trigger ingestion on demand
Spam filters	<code>lib/data-ingestion/spam-filter.ts</code> , <code>lib/security/spam-check.ts</code>	Crowdsourced reports stay clean

How to explain this in a viva: "Security here isn't about protecting credit cards, it's about protecting *the channel*. During a disaster the system is load-bearing: a DDoS, an injection attack, or a cross-district data leak is not a compliance incident, it's lives. So we rate-limit every sensitive endpoint, scope every query and every AI response to a district, verify webhook signatures, and audit every AI tool call — cheap controls that keep the platform trustworthy exactly when it must be."

13 Third-Party Services & Integrations

Service	Use	Verified usage
Supabase	Postgres/PostGIS/pgvector, Auth, Storage, Realtime	yes <code>real</code> — <code>lib/supabase/*</code> , migrations △ See Known Gaps (#7)
Twilio	SMS + WhatsApp inbound/outbound	yes <code>real</code> — <code>lib/alerts/twilio-client.ts</code> , <code>lib/sms/*</code>

web-push (VAPID)	Browser notifications	yes real – <code>lib/alerts/web-push.ts</code> , <code>worker/index.js</code>
ElevenLabs / Azure TTS / Google TTS	Alert voice	yes real – <code>lib/tts/providers.ts</code>
Nodemailer	Email strategy (FM studio)	yes real – <code>lib/broadcast/strategies/email-studio.ts</code>
basic-ftp	FTP strategy (FM)	yes real – <code>lib/broadcast/strategies/ftp-drop.ts</code>
OpenRouter / Groq / Bluesminds	LLM chain	yes real – <code>lib/ai/openrouter.ts</code>
DeepSeek	RAG chat model (OpenAI-compatible base URL)	yes real – <code>lib/rag/openai-client.ts</code>
WebLLM (@mlc-ai/web-llm)	On-device Gemma-2B	yes real – <code>lib/ai-bridge/webllm-provider.ts</code>
OSRM public / GraphHopper / openrouteservice	Routing	yes real – <code>lib/map/routing.ts</code> , <code>smart-routing.ts</code> (OPENROUTE: 79 refs, OSRM: 21 refs)
GLOFAS (ECMWF CDS)	Global flood forecasts	partial partial – settings UI + <code>lib/settings/integrations-settings.ts</code>
OpenWeatherMap	Rainfall ingestion	yes real – <code>lib/data-ingestion/fetcher.ts</code>
USGS / GDACS / EONET / FIRMS / GWIS	Multi-hazard feeds	configured; no active adapter for most env-only
IMD / CWC / NASA POWER / SentinelHub / GEE / HDX / WorldPop / OpenCage / Novu / SerpAPI / NewsData / Notion / HF / Crawl4AI / OpenTopoData /	Env-declared	not wired into code paths (0 refs for most; <code>NOVU: 0</code>) env-only

14 Testing & CI/CD

Unit tests (Vitest 4.1)

- `vitest.config.ts`: node env, excludes `.opencode/**`, `.next/**`, `app/**`.
- **137+ tests** across `lib/` (~120 `*.test.ts` files) — heavily covering: AI provider chain (`lib/ai/openrouter.test.ts`, including leaked-key regression tests), AIBridge, broadcast/FM (dispatcher, auto-trigger, IVR fallback, RDS encoder, compliance), CAP builder/hash, security (rate-limit, rate-limiter, sanitize, otp, require-role, data-isolation), offline-sync (db, sync-engine, quota, eviction, conflict, model-integrity), i18n, maps (routing, smart-routing, offline, citizen views), settings, allocation optimizer, crowdsourced accuracy, spam filters.

E2E (Playwright)

- `playwright.config.ts`: `testDir ./tests`, 180 s timeout, Chromium, auto-starts `npm run dev`.
- `tests/critical-path.spec.ts`: 7 flows — guest → command-center flood layer, public dashboard, alerts feed, AI planner chat, map shelters (+2).

CI/CD & tooling

Workflow	File	Steps
CI	<code>.github/workflows/ci.yml</code>	lint + <code>tsc</code>
APK build	<code>.github/workflows/build-apk.yml</code>	Capacitor build, JDK 17, API 36; commits <code>public/safesphere.apk</code>
Cron	<code>vercel.json</code>	<code>/api/cron/ingest</code> daily at <code>0 0 * * *</code>
Scripts	<code>package.json</code>	<code>demo:reset</code> , <code>test:ingestion</code> , <code>eval:retrieval</code> , <code>audit:env</code> , <code>smoke:agents</code> , <code>smoke:chat</code> , <code>format</code> , <code>format:check</code>

Other dev tooling: `opencode.json` (MCP `stitch` server for AI-assisted dev), `.opencode/` plugin package, Prettier + ESLint (eslint-config-next 14.2.35), `fake-indexeddb` for Dexie tests.

15 Native Android App (`native-android/`)

A second, fully native Kotlin client — built for offline-hardened field use, produced as `public/safesphere.apk` by the CI workflow. Package `com.safesphere.nativeapp`.

Layer	Contents
Database	Room: <code>SafeSphereDatabase.java</code> , <code>Converters.java</code> ; 14 entities (Alert, AuditLog, DistrictConfig, Evacuation, FamilyMember, InventoryMovement, KnowledgeDoc, Report, Resource, RoadClosure, Shelter, SosEvent, User) with matching DAOs + repositories
Sync	<code>sync/SyncWorker.java</code> — WorkManager-based sync to the backend
Seed data	<code>SeedDataLoader.java</code> + 12 JSON assets (<code>seed_alerts.json</code> , <code>seed_shelters.json</code> , <code>seed_users.json</code> , <code>seed_sos_events.json</code> , ...)
UI — admin	AdminDashboard, Analytics, AuditLogs, BulkOperations, DistrictConfig, SystemHealth, UserManagement fragments
UI — gov	AgentOrchestration, AiPlanner, Alerts, CommandCenter (overview/live-feed/resources/shelters/timeline/analytics), Evacuations (kanban + map), GovDashboard, Inventory, KnowledgeBase, SheltersMgmt, Triage
UI — citizen	CitizenDashboard, FamilyCircle, PublicAlerts, PublicShelters, Sos
Utilities	<code>ConnectivityMonitor.java</code> , <code>RoleManager.java</code> , <code>MainActivity.java</code> + nav graph + bottom nav (46 layout XMLs)

Related: `web/index.html` (standalone PWA entry used by Capacitor), `capacitor.config.ts` (appId `com.safesphere.app`, `webDir "web"`), server `https://disaster-link-ai.vercel.app`), `lib/native/capacitor-storage.ts` (bridge), `lib/pwa/` (wake-lock, pwa-utils).

16 Service Worker & PWA Internals

- **Generator:** `next-pwa 5.6.0` with `customWorkerDir: "worker"` (`next.config.mjs`) — the custom worker in `worker/index.js` augments the generated Workbox worker.
- **Offline shell:** cache `bharat-shakti-shell-v1`; document fallback routes to `/~offline`; runtime caching for API routes is network-first (fresh alerts win, cache as backup).
- **Push receiver:** VAPID push events surface as notifications (`lib/alerts/web-push.ts` provides the VAPID keys; `server/services/push-notifier.ts` sends + prunes subscriptions).
- **Background sync:** periodic sync relays queued Dexie mutations to the server (`lib/offline-sync/sw-sync.ts`, `sync-engine.ts`).
- **In-browser AI:** WebLLM Gemma-2B runs in `lib/ai-bridge/ai.worker.ts` + `worker-provider.ts`, model binary stored in Dexie `gemmaModel` with integrity checks.

17 Setup & Demo

Option A — Zero-setup Guest/Demo mode (best for the viva)

1. Visit `https://disaster-link-ai.vercel.app` (or local build at `localhost:3000`).
2. Choose **Guest / Demo mode** — `app/actions/auth.ts` sets the `guest_mode` cookie; `middleware.ts` admits guests with demo-scoped data.
3. Seeded Patna scenario from `lib/demo/seeder.ts` + `lib/mock-data/*`; reset anytime via `lib/demo/reset-scenario.ts`.
4. AI chat works with no LLM keys — the chain degrades to the rule-based fallback (correct, template-y answers).

Option B — Full local run

```
# 1. Dependencies (prisma generate runs automatically post-install)
npm install

# 2. Environment — copy .env.example → .env.local, fill in:
#   SUPABASE URL/keys, DATABASE_URL + DIRECT_URL, and any LLM/Twilio/VAPID keys you want live
cp .env.example .env.local
```

```

# 3. Database schema
npx prisma migrate dev          # applies migrations 0001-0029 (0018 is missing from the repo)

# 4. Seed demo data
npm run demo:reset

# 5. ML service (optional – /api/predict falls back to mock labels if absent)
cd ml_service
pip install -r requirements.txt
uvicorn api:app --host 127.0.0.1 --port 8000  # optional: python train_model.py to (re)train

# 6. Run the app
npm run dev                        # Next.js dev server (Turbopack)

# 7. Tests
npm test                          # Vitest (137+ unit tests)
npm run test:e2e                  # Playwright critical-path suite

```

What the demo depends on (and what happens when it's missing)

Piece	If missing	Degradation
LLM keys	Chat falls back through the chain	<code>rule-based-fallback.ts</code> answers (correct, generic)
OpenAI embedding key	Mock vectors	RAG silently becomes keyword search (<code>lib/rag/embeddings.ts</code>)
ML service (port 8000)	Mock/fallback labels	<code>lib/ml-client.ts</code> 3 s timeout → fallback path
Twilio / VAPID keys	SMS/push logged, not sent	Channel stubs keep the UI flow intact
FM station endpoints	<code>testMode</code> dry-run reports	<code>app/api/broadcast/fm/dispatch/route.ts</code> supports <code>testMode: true</code>

How to explain this in a viva: "The demo is designed to survive a live panel with zero API keys — guest mode seeds a full Patna scenario, and every external dependency degrades gracefully: the LLM chain bottoms out at a rule-based engine, RAG drops to keyword search without embedding keys, and the ML service's absence is handled by a

fallback in the client bridge. We deliberately built the demo so the worst case is 'boring but correct' instead of 'broken'."

18 Anticipated Viva Q&A (15 questions)

Q. "So what does your project actually do?"

Easy — improvise from the pitch

Section 1 pitch, verbatim. One line: predict → plan → reach everyone through every working channel, offline-first.

Q. "Why not just call one LLM directly instead of LangGraph?"

Needs memorizing — specific numbers/names involved

Deterministic graph for safety-critical decisions (risk labels, plan validation) so they can't be hallucinated; state is inspectable at each node (`lib/agents/graph.ts`); the LLM is layered only where open-ended reasoning is needed (chat + tools).

Q. Follow-up: where is the human in this loop?

The graph intentionally stops at `pending_approval`.

`app/api/agents/orchestrate/route.ts` documents this Human-in-the-Loop design ("The graph intentionally stops at `pending_approval`...") and exposes `requiresApproval`; the `communicatorNode` fires alerts only after a commander approves. Nothing is broadcast without a human decision.

Q. Follow-up: who decides the risk label — the LLM or the code?

The code. `predictorNode` computes the risk label by keyword/feature matching in `lib/agents/nodes/intelligence-nodes.ts`, not by an LLM call — the same input always yields the same label, which is exactly why it can be audited (see the Honesty note in Section 8.1).

Q. "How do you prevent the AI from hallucinating wrong emergency advice?"

Needs memorizing — specific numbers/names involved

Tools that can only see real DB data (`lib/ai/tools/`), RAG context injected as

`{SOP_CONTEXT}`, district scoping enforced in `app/api/chat/route.ts`, and every tool call logged to `ToolCallLog`.

Q. Follow-up: what if a tool finds no data — can the model invent numbers?

Honest answer: the tools return real DB rows when they exist, but two tools explicitly fall back to hard-coded demo data on an empty or unreachable DB —

`getFloodPrediction` returns a fixed CRITICAL / 120 mm mock (`lib/ai/tools/flood-tools.ts`) and `getShelters` returns 3 mock shelters (`lib/ai/tools/shelter-tools.ts`). In the demo, the "real data" the model quotes can therefore itself be mock. The guards are that every tool call is logged to `ToolCallLog` and the LLM never decides a risk label. Say this before the panel does.

Q. Follow-up: can you audit what the model actually did?

Yes — every tool call is recorded in the `ToolCallLog` table (`lib/security/tool-logger.ts`), and the deterministic pipeline state is inspectable at every node and stage (Section 8.1).

Q. "What happens if all your LLM providers are down at once?"

Needs memorizing — specific numbers/names involved

Groq → OpenRouter → Bluesminds (`lib/ai/openrouter.ts`), then on-device Gemma via WebLLM, then the 61-rule fallback (`lib/ai-bridge/rule-based-fallback.ts`). Alerting never depended on an LLM anyway.

Q. Follow-up: what exactly does that 61-rule fallback answer?

Pre-written static responses keyed by trigger phrases — substring matching on a normalized prompt, e.g. "flood" hits a step-by-step flood-safety checklist (move to high ground, don't walk or drive through flowing water, stay away from electricity, don't drink flood water). Unmatched prompts get a guided fallback pointing to the emergency helpline and the Resources tab — never a dead end (`lib/ai-bridge/rule-based-fallback.ts`).

Q. Follow-up: does an LLM outage ever block an alert?

No — the alerting path is fully deterministic (`lib/broadcast/auto-trigger.ts`),

Section 10); only conversational chat degrades through the chain.

Q. "Why XGBoost, not a neural network?"

Easy — improvise from the pitch

Small tabular problem (4 features), CPU training in seconds, interpretable feature importances — and defensible: an officer can see *why* a district escalated.

Q. Follow-up: what's the model's accuracy / F1 score?

Not determinable from this repo. `ml_service/train_model.py` prints test accuracy and a classification report to stdout at training time, but the metrics are not saved anywhere in the codebase. Before the viva: run `ml_service/train_model.py` (needs the `ML_DATABASE` historical CSV, ≥ 20 rows) and note the printed "Test accuracy" and report.

Q. Follow-up: what features does it use, and how explainable is it?

Four engineered features: `cumulative_rainfall_72h`, `river_level_trend`, `soil_saturation_index`, `elevation_m`. The training script prints feature importances, and the API returns class probabilities alongside the label (`ml_service/api.py`) — so "why did the district go to High?" is answerable.

Q. "How does RAG know which documents are relevant?"

Needs memorizing — specific numbers/names involved

Cosine similarity over pgvector embeddings (`<=>`, HNSW), top-k chunks injected into the prompt; keyword fallback if embeddings are unavailable

(`lib/retrieval/retrieve.ts`).

Q. Follow-up: what happens with no embedding key, or if the vector search fails?

`getEmbedding` returns null without `OPENAI_API_KEY`, and retrieval then falls back to keyword search over title/content (`lib/retrieval/retrieve.ts`, limit 4). If the vector query throws, the same keyword fallback catches it; if both fail, the call returns `[]` and `buildKnowledgeContext` returns `""` — the planner still runs on tool data. Retrieval degrades gracefully, it never throws.

Q. Follow-up: how many chunks actually go into the prompt?

Top-k defaults to 3: `lib/rag/vector-search.ts` uses `LIMIT 3` and `buildKnowledgeContext` passes limit 3; documents are chunked at 1000 chars with 200-char overlap (`lib/rag/chunker.ts`).

Q. "What was the hardest technical decision in this project?"

Easy — improvise from the pitch

Honestly: deciding *where the LLM should not be*. The tempting hackathon move is to let the LLM do everything; we constrained it to chat and made the agent pipeline deterministic so critical decisions are auditable. Second-hardest: the multi-channel fan-out — coordinating CAP generation, five FM strategies, retries, and a human-in-the-loop approval gate into one flow (`lib/broadcast/`).

Q. "What would you do differently with more time?"

Needs memorizing — specific numbers/names involved

Wire the declared-but-unused integrations (IMD/CWC feeds, GLOFAS adapter — env-config only today), commit the trained model artifact (`ml_models/` is just `.gitkeep`), restore migration `0018`, replace mock embeddings with a real key path, and move from in-memory rate limiting to a shared store (Redis) for multi-instance scaling.

Q. "How does this scale beyond one district/demo dataset?"

Easy — improvise from the pitch

The schema is district-keyed and middleware scopes by district; PostGIS + pgvector scale with indexes. Real scaling work: shared rate limiting, filling in the ingestion adapter layer, and periodic ML retraining with more districts' data (`train_model.py` handles any row count ≥ 20).

Q. "What's mocked/simulated vs actually real in this demo?" (be honest)

Needs memorizing — specific numbers/names involved

Real: the full web product, Prisma schema + migrations, the deterministic agent pipeline, FM dispatch logic (with `testMode`), TTS chain, Twilio + VAPID code paths, 137+ unit tests. Simulated/mock: risk labels without the ML service running; embeddings without a key (mock vectors); RLS is enforced application-side (`data-isolation.ts`, `require-role.ts`) with SQL RLS policies in migrations but primarily exercised through the app; FM station endpoints and hydrology data are seeded/mock (`lib/fm/mock-stations.ts`, `lib/mock-data/`, `lib/demo/seeder.ts`); most

ingestion env keys (IMD, CWC, GLOFAS, FIRMS...) are configured but only OpenWeather is wired in (`lib/data-ingestion/fetcher.ts`).

Q. "Why is everything district-scoped?"

Easy – improvise from the pitch

Real emergency response is run by district authorities (DDMA structure); scoping every query and AI response by district is both the security model (`data-isolation.ts`) and the mental model of the product (demo = Patna).

Q. "How did you test this?"

Needs memorizing – specific numbers/names involved

137+ Vitest unit tests across (`lib/`), Playwright critical-path E2E (`tests/critical-path.spec.ts` – 7 flows), CI running lint + (`tsc`) (`.github/workflows/ci.yml`), plus an APK build pipeline for the Capacitor/native app.

Q. "Why does the app work offline – that's unusual."

Easy – improvise from the pitch

Because connectivity is the first casualty of a disaster; Dexie/IndexedDB + service worker + background sync (`lib/offline-sync/`) make writes succeed locally and reconcile later. If the app needed internet to report a flood, it would fail exactly when needed most.

Q. "Your README says five AI agents – where are they?"

Needs memorizing – specific numbers/names involved

The five personas exist as pipeline stages, implemented as deterministic nodes (predictor/planner/allocator/validator/communicator) rather than autonomous LLM agents – an intentional reliability decision. The LLM powers the conversational planner instead. Say this *before* the panel asks – it's the project's best "we thought about this" story.

Q. "Why FM radio broadcast?"

Easy – improvise from the pitch

The cheapest device that works in a flood is a transistor radio. FM reaches people with no phone, no data, and no electricity for charging – the segment every other channel misses. It's also the only channel with a real approval workflow (auto-approve after 3 minutes), which makes a great human-in-the-loop story.

19 Dependency Inventory

Runtime dependencies (package.json)

Package	Version	Purpose
next	14.2.35	App Router framework
react / react-dom	^18	UI
ai	^7.0.55	AI SDK (streaming, tools)
@ai-sdk/openai / @ai-sdk/react	^4.0.33 / ^4.0.58	Provider + React helpers
openai	^7.4.0	OpenAI client (RAG/chat)
@langchain/core · langgraph · openai · textsplitters · langchain	1.x	Agent graph + chunking
@mlc-ai/web-llm	^0.2.84	On-device Gemma
@prisma/client	^6.19.3	ORM
@supabase/ssr / @supabase/supabase-js	^0.12.4 / ^2.112.1	Backend
@capacitor/core · android · cli	^8.5.0	Android wrapper
maplibre-gl / react-map-gl	^4.7.1 / ^7.1.9	Maps
@mapbox/mapbox-gl-draw / @turf/turf	^1.5.1 / ^7.4.0	Draw + geometry
dexie / dexie-react-hooks	^4.4.4 / ^4.4.0	Offline IndexedDB
next-pwa	^5.6.0	Service worker
twilio / web-push	^6.0.2 / ^3.6.7	SMS/WhatsApp + VAPID push
nodemailer / basic-ftp	^9.0.5 / ^6.2.0	FM email + FTP strategies

@tanstack/react-table / recharts	^8.21.3 / ^3.10.1	Tables + charts
framer-motion / lucide-react	^13 / ^1.30	Animation + icons
react-hook-form + @hookform/resolvers + zod	^7.84 / ^5.7 / ^4.4	Forms + validation
react-hot-toast / react-markdown + remark-gfm	^2.6 / ^10.1 / ^4.0	Toasts + markdown
qrcode.react / html5-qrcode	^4.2 / ^2.3	QR generate + scan
papaparse / pdf-parse	^5.5 / ^2.4	CSV + PDF ingestion
swr / next-themes / react-swipeable	^2.5 / ^0.4.6 / ^7.0	Polling + theme + gestures
class-variance-authority · clsx · tailwind-merge	—	UI utilities
tailwindcss / postcss	3.4 / 8	Styling
daisyui	^4.12.24	Installed, unused unused

Dev dependencies

@capacitor/assets, @playwright/test, @tailwindcss/typography, @types/* (mapbox draw, node, nodemailer, papaparse, react, react-dom, web-push), eslint 8, eslint-config-next 14.2.35, eslint-config-prettier, fake-indexeddb, prettier, prisma, tailwindcss-animate, tsx, typescript 5, vitest 4.1.10.

20 Environment Variables (`.env.example` → usage)

Variable	Used?	Notes
NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY	Yes	lib/supabase/client.ts, server.ts (service role = server-only)

<code>DATABASE_URL</code> , <code>DIRECT_URL</code>	Yes	Prisma pooler + migrate pair
<code>OPENWEATHER_API_KEY</code> , <code>GLOFAS_CDS_API_KEY</code>	Yes / Partial	fetcher.ts / settings UI
<code>NASA_API_KEY</code> , <code>GEE_SERVICE_ACCOUNT_JSON</code> , <code>SENTINELHUB_*</code>	No	env-only
<code>USGS_*</code> , <code>GDACS_*</code> , <code>NOAA_*</code> , <code>FIRMS_API_KEY</code> , <code>GWIS_API_URL</code> , <code>EONET_API_URL</code>	Partial	configured; no active adapters
<code>IMD_*</code> , <code>CWC_NWDP_*</code> , <code>DATA_GOVIN_API_KEY</code>	No	env-only
<code>OSRM_PUBLIC_URL</code> , <code>GRAPHHOPPER_*</code> , <code>OPENROUTE_API_*</code>	Yes	<code>lib/map/routing.ts</code> , <code>smart-routing.ts</code>
<code>WORLDPOP_*</code> , <code>HDX_*</code>	No	env-only
<code>OPENCAGE_API_KEY</code> , <code>NOMINATIM_URL</code> , <code>OPENTOPODATA_API_URL</code> , <code>OPENAQ_API_URL</code>	No	env-only
<code>GROQ_API_KEY(+BACKUP)</code> , <code>OPENROUTER_API_KEY(+BACKUP)</code> , <code>BLUESMINDS_API_KEY</code>	Yes	<code>lib/ai/openrouter.ts</code> , <code>model-provider.ts</code>
<code>SERPAPI_API_KEY</code> , <code>NEWSDATA_API_KEY</code> , <code>NOTION_*</code> , <code>HF_TOKEN</code> , <code>CRAWL4AI_*</code> , <code>REST_COUNTRIES_API_KEY</code>	No	env-only
<code>TWILIO_ACCOUNT_SID</code> , <code>TWILIO_AUTH_TOKEN</code> , <code>TWILIO_FROM_NUMBER</code>	Yes	<code>lib/alerts/twilio-client.ts</code>
<code>NEXT_PUBLIC_VAPID_PUBLIC_KEY</code> , <code>VAPID_PRIVATE_KEY</code> , <code>VAPID_SUBJECT</code>	Yes	<code>lib/alerts/web-push.ts</code>
<code>NOVU_API_KEY</code> , <code>NOVU_APPLICATION_ID</code>	No (0 refs)	env-only
<code>NEXT_PUBLIC_SITE_URL</code> , <code>CRON_SECRET</code> , <code>ML_SERVICE_URL</code>	Yes	fetchers / cron auth / ml-client
<code>ELEVENLABS_API_KEY</code> , <code>ELEVENLABS_VOICE_ID</code> , <code>AZURE_TTS_KEY</code>	Yes	<code>lib/tts/providers.ts</code>

`AZURE_TTS_REGION`, `GOOGLE_TTS_API_KEY`

`FM_BROADCAST_TOKEN`, `FM_FTP_*`,

Yes

`lib/broadcast/strategies/*`

`FM_SMTP_*`, `FM_EMAIL_TO`

21 Known Gaps, Discrepancies & Assumptions

1. **README vs code — "AI agents":** README describes five named LLM agents; code implements a deterministic LangGraph (predictor → planner → allocator → validator → communicator) with rule-based nodes. Chat uses the LLM + tools; the graph does not.
2. **Model count:** README says 28 models; `prisma/schema.prisma` has 33.
3. **Missing migration:** `supabase/migrations/0018` absent (0017 → 0019).
4. **Unused deps/keys:** `daisyui`, `NOVU_*`, `NOTION_*`, `HF_TOKEN`, `CRAWL4AI_*`, `WORLDPOP_*`, `HDX_*`, `OPENCAGE_API_KEY`, `OPENTOPODATA_API_URL`, `OPENAQ_API_URL`, `SERPAPI_API_KEY`, `NEWSDATA_API_KEY`,
IMD/CWC/NASA/GEE/SentinelHub vars — declared, unreferenced.
5. **Embedding fallback:** `lib/rag/embeddings.ts` + `vector-search.ts` return mock vectors/results without keys — RAG silently degrades to keyword search.
6. **ML model artifact:** `flood_xgboost_model.pkl` expected at runtime; `ml_models/` holds only `.gitkeep`. Risk label vocab differs between API (`Low/Medium/High/Critical`) and app (`Safe/Watch/Warning/Evacuate`) — mapped in `lib/ml-client.ts`.
7. **Realtime:** Supabase transport implemented; demo uses the mock transport — live collaboration requires `useMockRealtime` disabled.
8. **Rendering:** PWA pages rely on client components + `force-dynamic`; offline behavior is IndexedDB-based, not server-rendered.
9. **Role enum drift:** `middleware.ts` tolerates a legacy `viewer` role not present in the Prisma enum.
10. **Ingestion scope:** only OpenWeather has an active adapter in `lib/data-ingestion/fetcher.ts`; other feeds appear in settings UI only.
11. **`data_ingestion/` folder:** contains only `.gitkeep`; real ingestion lives in `lib/data-ingestion/`.
12. **ML training data:** `ML_DATABASE/` holds ~50 PDFs + a notebook as the corpus; the model itself is trained offline (`train_model.py`).

Term	One-line meaning
RAG	Retrieval-Augmented Generation — fetching relevant documents and pasting them into the LLM prompt before answering
LLM	Large Language Model — the AI that generates text (here: Groq/OpenRouter/Bluesminds/DeepSeek)
LangGraph	LangChain's graph framework for inspectable multi-step LLM/agent pipelines
Agent	A pipeline stage with a job (in this codebase: deterministic nodes, not autonomous bots)
RBAC	Role-Based Access Control — permissions tied to roles (public, field_responder, district_admin, super_admin)
RLS	Row-Level Security — Postgres policies limiting visible rows; here complemented by app-level <code>data-isolation.ts</code>
CAP	Common Alerting Protocol — the XML standard format for emergency alerts (<code>lib/cap/</code>)
RDS (radio)	Radio Data System — the text/data channel sent alongside FM audio (<code>rds-encoder.ts</code>)
IVR	Interactive Voice Response — automated phone calls (FM escalation, <code>fm-ivr-fallback.ts</code>)
PostGIS	Postgres extension for geospatial types and queries (polygons, <code>ST_Intersects</code> , distances)
pgvector	Postgres extension for vector similarity search (cosine <code><=></code> + HNSW)
HNSW	Hierarchical Navigable Small World — graph index for fast approximate nearest-neighbor search
OSRM / GraphHopper / openrouteservice	Routing engines for directions and isochrones (<code>lib/map/routing.ts</code> , <code>smart-routing.ts</code>)
VAPID	Voluntary Application Server Identification — the key protocol for Web Push

PWA	Progressive Web App — installable web app with service worker, offline support, push
Service worker	Background browser script for caching, syncing, and pushes (<code>worker/index.js</code>)
Dexie	Typed wrapper over IndexedDB for offline storage
Workbox / next-pwa	Libraries generating service-worker caching strategies
WebLLM	(<code>@mlc-ai/web-llm</code>) — runs small LLMs (Gemma 2B) in the browser/on-device
XGBoost	Gradient-boosted decision-tree library used by (<code>ml_service/</code>)
FastAPI	Python web framework hosting the ML model (<code>ml_service/api.py</code>)
OTP	One-Time Password — the passwordless login method (<code>lib/security/otp.ts</code>) via GetOTP)
SOP	Standard Operating Procedure — emergency documents ingested into RAG
Dedup	De-duplication — the 6-hour window preventing repeat alert spam
SMS/WhatsApp webhooks	Signature-verified inbound endpoints for text/WhatsApp commands
TTS	Text-to-Speech — alert voice generation (ElevenLabs → Azure → Google)
Capacitor	Tool that wraps the web app as a native Android app (<code>capacitor.config.ts</code>)
CRON	Scheduled jobs — Vercel cron hits (<code>/api/cron/ingest</code>) daily
GeoJSON	JSON format for map geometries (flood zones, shelters) used by MapLibre
Room	Android SQLite ORM used by the native Kotlin app (<code>native-android/</code>)
DAO	Data Access Object — Room interface for table queries (14 DAOs in native app)
SWR	Stale-While-Revalidate — React data-fetching hook used for dashboard polling

End of guide. All file paths relative to repository root · Verified against source on 20 Aug 2026.
Anything you'll be quizzed on, verify against the cited files — the repo rewards reading it.